Mergesort bottom-up

Material de estudo · trabalho Mergesort × Quicksort

1. A ideia em um minuto

O Mergesort **bottom-up** (de baixo para cima) ordena sem dividir nada e sem recursão. Ele parte de um fato simples: **um bloco de 1 elemento já está ordenado**. Então ele intercala blocos vizinhos de 1 e forma blocos de 2; intercala blocos de 2 e forma blocos de 4; depois 8, 16... A largura do bloco dobra a cada **passada**, até um bloco cobrir o vetor inteiro.

> **No dia a dia:** um torneio de chaves. Primeiro jogam as duplas, depois os vencedores formam quartetos, depois grupos de 8, até sobrar um campeão. Ninguém precisa "dividir" o torneio antes: ele já começa pelas duplas.

Intercalar (merge) é juntar dois blocos já ordenados em um só: compara o primeiro de cada bloco, copia o menor para um vetor auxiliar e avança naquele bloco. Quando um bloco acaba, copia o resto do outro.

2. Passo a passo com os 8 números dos slides

Passo	Largura	Pares intercalados	Vetor depois da passada	Blocos
início	–	–	`[38] [27] [43] [3] [9] [82] [10] [15]`	8 de 1

1	1	`[38] + [27]`
		`[43] + [3]`
		`[9] + [82]`
		`[10] + [15]`
			`[27 38] [3 43] [9 82] [10 15]`	4 de 2

2	2	`[27 38] + [3 43]` `[9 82] + [10 15]`	`[3 27 38 43] [9 10 15 82]`	2 de 4
3	4	`[3 27 38 43] + [9 10 15 82]`	`[3 9 10 15 27 38 43 82]`	1 de 8

São 3 passadas para 8 elementos, porque $\log_2 8 = 3$. Para 300.000 elementos são 19 passadas.

3. E quando o tamanho não é potência de 2?

O último bloco pode ficar menor, ou ficar sem par. O código trata os dois casos: `fim = min(ini + 2*largura, n)` corta o bloco no fim do vetor, e um bloco sem par simplesmente não é intercalado naquela passada (já está ordenado) e entra na próxima. Exemplo com 5 números:

Passo	Largura	Pares intercalados	Vetor depois da passada	Observação
início	–	–	`[5] [1] [4] [2] [3]`	5 blocos de 1
1	1	`[5] + [1]` `[4] + [2]`	`[1 5] [2 4] [3]`	o [3] fica sem par
2	2	`[1 5] + [2 4]`	`[1 2 4 5] [3]`	o [3] ainda sem par
3	4	`[1 2 4 5] + [3]`	`[1 2 3 4 5]`	agora o [3] entra

4. O código (exatamente o que está em src/benchmark.cpp)

```
// ---- Mergesort BOTTOM-UP (iterativo, sem recursão)
//      Começa com blocos de 1 elemento (já ordenados) e intercala blocos vizinhos:
//      largura 1 -> blocos de 2, largura 2 -> blocos de 4, ... até cobrir o vetor.
//      São ceil(log2 n) passadas; cada passada percorre os n elementos: O(n log n).

// Intercala v[ini..meio) e v[meio..fim), os dois já ordenados, usando aux.
static void intercalar(vector<int>& v, vector<int>& aux, int ini, int meio, int fim) {
    int i = ini, j = meio, k = ini;
    while (i < meio && j < fim) aux[k++] = (v[i] <= v[j]) ? v[i++] : v[j++];
    while (i < meio) aux[k++] = v[i++];
    while (j < fim) aux[k++] = v[j++];
    for (int t = ini; t < fim; t++) v[t] = aux[t];
}

void mergesort(vector<int>& v) {
    int n = (int)v.size();
    vector<int> aux(n);
    for (int largura = 1; largura < n; largura *= 2) {          // 1, 2, 4, 8, ...
        for (int ini = 0; ini < n - largura; ini += 2 * largura) {  // pares de blocos vizinhos
            int meio = ini + largura;                               // início do 2º bloco
            int fim = min(ini + 2 * largura, n);                    // o último bloco pode ser menor
            intercalar(v, aux, ini, meio, fim);
        }
        // Um bloco que sobra sem par já está ordenado e entra na próxima passada.
    }
}
```

Trecho	O que faz
`vector<int> aux(n)`	Vetor auxiliar do mesmo tamanho: é a “mesa extra” onde a intercalação monta o resultado. Memória O(n).
`for (largura = 1; largura < n; largura *= 2)`	Uma volta por passada: largura 1, 2, 4, 8… São $\lceil \log_2 n \rceil$ voltas.
`for (ini = 0; ini < n - largura; ini += 2*largura)`	Anda de par em par de blocos. A condição garante que o 2º bloco existe; se não existir, o bloco sobra.
`meio = ini + largura`	Onde começa o 2º bloco do par.
`fim = min(ini + 2*largura, n)`	Onde termina o 2º bloco; o min corta o último bloco no fim do vetor.
`(v[i] <= v[j]) ? v[i++] : v[j++]`	Dentro de intercalar: em empate pega o da esquerda. É isso que torna o algoritmo **estável**.

5. Bottom-up, top-down e Quicksort

Aspecto	Mergesort bottom-up	Mergesort top-down	Quicksort clássico (nosso)
Estratégia	Junta blocos de 1, 2, 4, 8…	Divide ao meio até 1, depois junta	Pivô = elemento do meio; separa menores e maiores
Recursão	Não (dois laços)	Sim, profundidade log n	Sim, nos dois lados
Tempo (melhor / médio / pior)	n log n / n log n / n log n	n log n / n log n / n log n	n log n / n log n / n^2
Memória extra	O(n): vetor auxiliar	O(n) + pilha O(log n)	O(log n) em média: só a pilha
Estável?	Sim	Sim	Não

Com 8 elementos (ou qualquer potência de 2), bottom-up e top-down fazem exatamente as mesmas intercalações; só a ordem em que acontecem muda. Com outros tamanhos os blocos podem sair diferentes, mas o custo é o mesmo.

6. O que medimos (média de 20 execuções, em ms)

Itens (n)	Mergesort bottom-up	Quicksort	Diferença	Mais rápido
30	0,0008	0,0008	5%	Mergesort
300	0,0116	0,0120	4%	Mergesort
3.000	0,1567	0,1545	1%	empate
30.000	2,26	2,06	9%	Quicksort
300.000	27,28	23,14	15%	Quicksort

Abaixo de 3.000 itens os tempos são de microssegundos e a porcentagem muda a cada rodada. Para ver as tabelas completas (média, desvio, mínimo, máximo e crescimento): PAINEL_ADM.bat, opção [11].

7. Perguntas que a banca pode fazer

Qual a diferença entre bottom-up e top-down?
O top-down é recursivo: divide o vetor ao meio até sobrar 1 elemento e junta na volta da recursão. O bottom-up pula a divisão: começa direto dos blocos de 1 e junta em laços, dobrando a largura.

Por que o custo é n log n?
Cada passada percorre os n elementos uma vez (intercalando). Como a largura dobra, são $\log_2 n$ passadas. n × log n. Para 300.000: 19 passadas.

Por que precisa do vetor auxiliar?
Para intercalar dois blocos sem perder valores: o resultado é montado no auxiliar e copiado de volta. É a “mesa extra” para juntar as cartas.

Ele é estável? Por quê?
Sim. Na intercalação, em caso de empate (`v[i] <= v[j]`) ele pega o elemento do bloco da esquerda, que veio antes. Itens iguais nunca trocam de ordem: como a fila do banco.

E se n não for potência de 2?
O último bloco fica menor ou sem par. O min() corta o bloco no fim do vetor, e o bloco sem par espera a próxima passada.

Se os dois são n log n, por que o Quicksort foi mais rápido?
A notação O esconde as constantes. O Mergesort copia cada elemento para o auxiliar e de volta em toda passada; o Quicksort troca elementos no próprio vetor e usa melhor a memória cache.

Quando vale mais usar o bottom-up?
Quando não se quer recursão (sem risco de estourar a pilha), em listas ligadas e em ordenação de arquivos grandes em disco, que são lidos em blocos. Cada passada também é fácil de dividir entre processadores.

8. Para estudar mais

- **Sedgewick & Wayne, Algorithms, 4ª ed., seção 2.2 (Mergesort)**: a referência do bottom-up (classe MergeBU), com explicação e análise. Conteúdo gratuito em algs4.cs.princeton.edu/22mergesort
- **Cormen et al., Algoritmos: Teoria e Prática (CLRS), seção 2.3**: a intercalação (MERGE) e a análise n log n.
- **Wikipedia (inglês), “Merge sort”, seção “Bottom-up implementation”**: pseudocódigo curto.